ACI-CO Repository Architecture

Colombian Actuarial Climate Index (ACI-CO) pipeline

August 20, 2026

Abstract

This document is a LaTeX companion to `ARCHITECTURE.md` and `CLAUDE.md` at the repository root. It describes how the ACI-CO codebase is organized: the scientific target it computes, the actual data flow between scripts (as opposed to the README's idealized version), where the repository's documentation is authoritative versus stale, and how to read already-processed outputs from a notebook (e.g. Google Colab) using the `aci_lib` data-access library introduced alongside this document. It is a description of the codebase as it stands, not a specification – where the code and the docs disagree, the code wins.

Contents

1 What this repository is

This is a research pipeline that builds the **Colombian Actuarial Climate Index (ACI-CO)** from ERA5(-Land) reanalysis data (1961–2024): a Colombian adaptation of the Actuaries Climate Index (ACI) framework, extended with ENSO calibration, a bootstrap uncertainty band, multi-dataset robustness checks, and validation against Colombia's national disaster registry (UNGRD).

It is an active research codebase, not a packaged library: there is no test suite, several parallel/experimental implementations of the same pipeline stage coexist deliberately, and a non-trivial share of the *working* pipeline was never committed to git (Section 7).

Authoritative methodology source. The methodology (how T90, T10, Rx5day, CDD and WP are computed; how the composite, ENSO decomposition, and bootstrap bands work) is described in `articles/aci_co_submission/article1.tex`, §3–§7 – *not* the older root-level `articles/article1.tex` (a superseded LOWESS-based draft), and not the empty stubs in `docs/methodology.md` / `docs/results.md` / `docs/zones_description.md`.

2 The scientific target: ACI-CO

The ACI-CO composite is the unweighted mean of five signed, standardized components (divided by $K = 5$; an earlier draft divided by 6 for a placeholder sea-level component, since dropped for lack of tide-gauge coverage):

Component	Meaning	Sign	Threshold
T90	Fraction of days/month with temperature above the 1961–1990 baseline 90th percentile	+	Empirical P90 per grid cell × calendar month
T10	Fraction of days/month with temperature below the baseline 10th percentile	−	Empirical P10
Rx5day	Max 5-day precipitation total per month, z-scored vs. 1961–1990 climatology	+	z-score vs. monthly climatology

Component	Meaning	Sign	Threshold
CDD	Annual max consecutive dry days (<1 mm), interpolated onto the monthly axis	+	—
WP	Wind-power ($\propto U^3$) exceedance	+	Empirical P90 (current); replaces an earlier parametric $\mu + 1.28\sigma$ threshold, discarded because WP is right-skewed

2.1 Extensions beyond the base ACI framework

- **ENSO-neutral decomposition** (§4): asymmetric ARIMAX regression per component/region on Niño-3.4 ($E^+ = \max(E, 0)$, $E^- = \min(E, 0)$) plus a linear trend and AR(p) errors, separating secular warming from ENSO-driven variability.
- **Bootstrap uncertainty bands** (§4.4): block-resample ($L = 12$ mo) the 1961–1990 baseline composite into $B = 500$ (national) / $B = 200$ (regional) null-model replicates, testing whether the observed trend exceeds baseline-period variability. Replaced an earlier LOWESS residual-envelope approach.
- **Multi-dataset robustness** (§5): ERA5-Land vs. ERA5 standard vs. CHIRPS.
- **UNGRD validation** (§7): negative-binomial regression of disaster counts (floods, landslides, windthrow, vegetation fires – 49,734 events, 1998–2022) on ACI-CO components.

2.2 Regions actually computed

Despite the paper conceptually discussing 5 natural regions and 32 departments, the implementation aggregates **4 units**: national (area-weighted, all of Colombia) and 3 departments chosen for insurance relevance – **Antioquia**, **Cundinamarca-Bogotá**, and **Valle del Cauca**. Shapefiles also exist for Bogotá, Medellín, Cali, San Andrés/Providencia, Pacífico, and Amazonas, but the corresponding `data/processed/anomalias_{bogota,medellin,san_andres_providencia}/` directories are empty – never populated.

3 Repository layout

Path	Contents
`src/scripts/`	The original OPT pipeline: download, percentile, and anomaly-calculation scripts (`calcular_*`, `ecmwf_descarga.py`, `unir_archivos.py`, `graficas.py`, ...). Spanish-language docstrings dominate here.
`src/forecast_-scripts/`	Two independent forecasting families (AutoTS ensembles vs. SARIMAX/ETS + ENSO) plus `common/`, the shared modules extracted from both (Section 4.2). English-language, newer code.
`src/dashboard/`	`app.py` – a read-only Streamlit dashboard that reads Stage 3/5 CSVs directly; not a pipeline stage.

Path	Contents
src/utils/	Shared helpers (get_available_years, get_cached_shapefile), consolidated from near-identical copies previously duplicated across the calcular_* scripts.
src/aci_lib/	Read-only data-access library for notebooks (Section 8) – added alongside this document, not part of the original OPT/forecast pipeline.
data/raw/, data/processed/	Pipeline inputs/outputs, entirely gitignored except for .gitkeep placeholders (Section 7).
articles/	Three papers at different stages: the current submission (aci_co_submission/), a superseded LOWESS draft (article1.tex), and a separate forecasting report (AnnualICAForecast_Report.tex).
docs/	Mostly empty stubs (methodology.md, results.md, zones_description.md); the one substantive file is Diccionario.md, a ∼2400-line data-source vetting document.
tests/	Placeholder only (test.txt); there is no test suite or CI.

4 Actual data flow

The README describes an idealized 8-step flow; several of the scripts it names do not exist (Section 6). The flow below is reconstructed from what the scripts actually read and write.

Stage	Script(s)	Output
0. Download	ecmwf_descarga.py (canonical), sst_descarga.py, get_aux_data.py, IDEAM station data via pyreadr	data/raw/era5/*.grib, SST GRIB, data/raw/auxiliary/, data/raw/ideam/*.rds
1. Merge / resample	unir_archivos.py; independently, the untracked run_aci_colombia.py re-merges raw GRIB into a from-scratch reimplementation	era5_daily_combined_{tmp,rain, wind}.nc; data/processed/aci_daily/, aci_colombia_1961_2024.csv
2. Percentiles (1961–1990 baseline)	calcular_percentil_- {temperatura,lluvia, viento}.py	era5_*_percentil.nc, percentiles*.csv
3. Anomalies, per region	calcular_anomalias_- {temperatura,lluvia, viento}.py, orchestrated by calcular_anomalias_- regiones.py over data/shapefiles/*.shp	data/processed/anomalias_- <region>/anomalies_{temperature, precipitation,drought, wind}_combined.csv (populated: antioquia, cali, colombia, cundinamarca_bogota, valle_cauca; empty: bogota, medellin, san_andres_providencia)

Stage	Script(s)	Output
4. Daily variants	`produce_daily_-anomalies.py`, `append_2025_2026.py` (untracked)	`data/processed/daily_-anomalies/*.xlsx`
5. ENSO / SST index	`anomalias_sst_daily.py`, `compare_enso_flags.py`	`sst_index_colombia_pacific.csv`, `enso_flag_comparison.csv`, validated against NOAA CPC `oni.ascii.txt`
6. Downscaling	`downscale_era5_-temperature.py`, `downscale_era5_tp.py` (RandomForest, DEM/NDVI/IDEAM)	`data/processed/downscaled/`, validated in `data/processed/era5_-ideam_comparison/`
7. Visualization / export	`graficas.py`, `generate_-correlation_matrices.py`	`articles/graficas/<region>/*.png`, xlsx exports, correlation heatmaps
8. Forecasting	Two independent families – see Section 4.2	`articles/graficas/forecast_*/`

`data/indices/` and `data/zones/` exist but are effectively empty – no script currently writes to those exact paths.

Raw ERA5 `.grib` backup. `data/raw/era5/*.grib` is gitignored (Section 7) and not reproduced anywhere in version control. A backup copy of these raw downloads is kept on Google Drive: https://drive.google.com/drive/folders/17SdbsxVJYnqjPzIlS4FFZ6MowfMcHfqW.

4.1 Two parallel "correctness check" implementations

There are effectively two independent implementations of the core ACI algorithm in this repository:

- **The OPT pipeline** (`src/scripts/calcular_*`, tracked in git, the README's documented flow) – the original project pipeline, per-region, percentile/anomaly based.
- `run_aci_colombia.py` (untracked) – a from-scratch reimplementation of the ACI-Python reference algorithm directly on raw GRIB, used to cross-validate the OPT pipeline's output. `compare_aci_-colombia.py` and `compare_repos.py` (also untracked) diff the two and document the algorithmic differences.

Neither is "the old version" of the other – they are independent implementations kept deliberately in sync for cross-validation, and this comparison grounds the paper's "validation against x-ACI" discussion (§8).

4.2 Forecasting: two families plus shared modules

Two independent forecasting families exist side by side and are *not* a superset/subset of each other:

- `forecast_ica_{monthly,daily}.py` – AutoTS ensembles.
- The `ETS(X)_*` family – statsmodels SARIMAX/ETS with an ENSO exogenous regressor.

`src/forecast_scripts/common/` holds logic that was byte-identical or near-identical across these scripts before consolidation:

Module	Extracted from	Used by
`data_loading.py`	Byte-identical in `forecast_ica_monthly.py` and `ETS_ica_forecast.py`	Both, via thin same-named wrapper methods
`climate_index.py`	Near-identical in all three monthly forecasters	All three
`stationarity.py`	Near-identical in `ETS_ica_forecast.py` / `ETSX_ica_forecast.py`	Both
`diagnostics.py` (`ModelErrorDiagnostics`)	Moved unchanged from the old `error_diagnostics.py`	Not currently called by anything

Deliberately *not* merged (structurally parallel, not literal duplicates): the `ONIDataHandler` /`ONIDataHandlerDaily` ENSO-fetching classes, the SARIMAX+Lasso fitting methods, the AutoTS setup (same shape, different hyperparameters), and all `visualize_forecasts`-style plotting methods (4 separate implementations, each with different overlays).

5 Documentation status

Document	Status
`docs/methodology.md`, `docs/results.md`, `docs/zones_description.md`	Empty stubs. Methodology lives in `articles/aci_co_submission/article1.tex` instead.
`docs/explicacion_excel.md`	One truncated sentence, no body.
`docs/data_dictionary.md`	Early, incomplete fragment (HUMBOLDT-only) of `docs/Diccionario.md`.
`docs/Diccionario.md`	The real, complete data-source vetting document (∼2400 lines, 20 candidate sources evaluated). Canonical over `data_dictionary.md`.
`README.md`	Describes an idealized 8-step flow; two referenced scripts do not exist.

6 Discrepancies between README/docstrings and reality

- README step 1 says to run `descargar_datos.py` – it does not exist; `ecmwf_descarga.py` is the actual entry point.
- README step 6 says to run `sealevel.py` – does not exist. `sealevel2.py` exists but does not do sea-level analysis; it is a Mann-Kendall trend + Q-Q normality check on precipitation-like series with leftover Jupyter cell markers, suggesting it was pasted from a notebook and misnamed. Sea level is not part of the current ACI-CO methodology.

- `src/scripts/earthub_descarga.py` has a whole-file indentation error and cannot run as-is, on top of embedding a live-looking credential (Section 7.1).
- `src/scripts/display.py` is not a plotting module – it is a Greece-precipitation download script (also embeds the same credential) that produced a 562 MB `.nc` file at the repo root as a side effect.
- Stale internal docstrings/comments: `era5datasets.py` says `"""compare_land_only.py"""`; `get_aux_data.py`'s header says `get_auxiliary_data.py`. Trust the filename/git path.
- `graficas_altaresolucion.py` is a 4-line dead stub.
- `ETSX_seasonal_motif_forecast.py` (the "broken two-stage Lasso→ETS" approach `ETSX_daily_ica_forecast.py`'s docstring says it replaced) has been deleted – confirmed unused before removal.
- `example_grid_search_era5.py` imports `ONIDataHandler` from `ETSX_ica_forecast.py` expecting methods that do not exist on that class – a pre-existing break, unrelated to the forecast_scripts modularization.
- `articles/AnnualICAForecast_Report.tex` references `scripts/daily_forecast_to_annual.py` and `scripts/ar1_vs_ets_final.py` – neither exists anywhere in the repository. Treat that report's pipeline as documented-but-unimplemented.

7 The git tracking gap

A significant portion of the *current, more-correct* pipeline exists only in the working tree and has never been `git add`ed: `run_aci_colombia.py`, `compare_aci_colombia.py`, `compare_repos.py`, `create_region_shapefiles.py`, `probe_2025.py`, `produce_daily_anomalies.py`, `append_2025_2026.py`, `reproduce.py`, `plot_repo_anomalies.py`, and most of `articles/` (the `aci_co_submission/` folder, `produce_paper_plots.py`, `build_validation_outputs.py`, etc.).

Conversely, things that *are* committed include generated junk (`presentation.aux/log/out`, `texput.log`, `output.log`, `comparison.png`, `storm_daniel.png`, `.vs/`, `proyecto-indices/`, root-level `.nc`/`.csv` data files) and a `.gitignore` that was itself accidentally overwritten with a PowerShell heredoc command instead of its output.

Untracked here often means "uncommitted work in progress," not "disposable." Do not delete untracked files without checking which side of this split they are on.

7.1 Known secrets in the repository

Two Earth Data Hub PAT tokens are hardcoded and **committed** in `src/scripts/display.py` (line 13) and `src/scripts/earthub_descarga.py` (line 15) – the same token, duplicated. A CDS API key also appears in `GRID_SEARCH_AND_ERA5_GUIDE.md` (line 44). These require rotation/history-scrubbing independent of any code change. New download scripts should follow the pattern in `ecmwf_descarga.py` / `sst_descarga.py`: `cdsapi.Client()` reads credentials from the user's local `~/.cdsapirc`, nothing embedded in source.

8 The `aci_lib` data-access library

`src/aci_lib/` is a small, dependency-light Python package that turns `data/processed/` into a browsable, loadable catalog, so that already-computed pipeline outputs can be pulled into a notebook (locally or in Google Colab) without knowing the directory's internal layout by heart.

8.1 Why it exists

`data/processed/` mixes single tabular files, single gridded NetCDF files, and long runs of per-year or per-year-month NetCDF files that really form one logical time series (e.g. 768 files under `anomalias_colombia/anomalies_temperature_1961_1.nc` through `_2024_12.nc`). `aci_lib` groups the latter automatically and exposes a uniform three-function API:

- `list_datasets()` – a DataFrame of every dataset name, kind, file count, and (where known) a description sourced from this document's data-flow tables.
- `describe(name)` – the underlying file list and kind for one dataset.
- `load(name)` – returns a `pandas.DataFrame` for CSV/XLSX datasets, an `xarray.Dataset` for single or multi-file NetCDF datasets (opened lazily via `xarray.open_mfdataset`), or a file path for images/raw text files.

8.2 Important caveat: data is not in git

`data/processed/` is entirely gitignored (Section 7) – only a `.gitkeep` placeholder is tracked. A plain `git clone` of this repository, including in Colab, brings the *code* in `src/aci_lib/` but *not* the data it reads. The data must be supplied separately: mount it from Google Drive, upload and unzip an archive of `data/processed/`, or copy it in via any other means available in the notebook environment.

8.3 Quickstart in Google Colab

```
# 1. Get the code (data/processed/ is gitignored, so this clone
# does NOT include the data -- only src/aci_lib/).
!git clone https://github.com/<org>/aca_indice_climatico_opt-main.git
import sys
sys.path.insert(0, "/content/aca_indice_climatico_opt-main/src")

# 2. Supply the data. Pick ONE of:
# (a) mount Drive, if data/processed/ was uploaded there beforehand:
from google.colab import drive
drive.mount("/content/drive")
DATA_DIR = "/content/drive/MyDrive/aca_data/processed"

# (b) upload + unzip a local copy:
# from google.colab import files
# files.upload() # e.g. processed.zip
# !unzip -q processed.zip -d /content/aci_data
# DATA_DIR = "/content/aci_data/processed"

import aci_lib
aci_lib.set_data_dir(DATA_DIR)

# 3. Browse and load.
catalog = aci_lib.list_datasets()
catalog.head(20)

national_composite = aci_lib.load(
    "anomalias_colombia/anomalies_temperature_combined"
) # -> pandas.DataFrame
```

```
gridded_series = aci_lib.load(
    "anomalias_colombia/anomalies_temperature"
) # -> xarray.Dataset, lazily concatenated from ~768 monthly files
```

Run locally, inside a checkout of this repository, `set_data_dir()` is unnecessary: `aci_lib.get_data_dir()` walks up from `src/aci_lib/` to find the repository root (marked by `CLAUDE.md`) and locates `data/processed/` on its own.

8.4 Scope

`aci_lib` currently covers everything under `data/processed/` only – it does not reach into `articles/*/graficas/paper_figures/` (the paper's final figures/tables) or `data/raw/`. Both are reasonable follow-up extensions if a notebook workflow needs them, but were kept out of scope to keep the catalog's grouping logic simple and its descriptions trustworthy.

9 Benchmarking the pipeline in Colab

Unlike `aci_lib` (read-only over already-computed `data/processed/`), this capability runs the actual pipeline *stages* – Stage 1 merge/resample, Stage 2 baseline percentiles, Stage 3 anomalies – against raw ERA5 `.grib`, to see how long they take on whatever compute a given Colab session provides.

9.1 Why no script changes were needed

`unir_archivos.py`, `calcular_percentil_temperatura.py`, and `calcular_anomalias_temperatura.py` already expose plain, parameterized functions (`process_yearly_data_tmp`, `calcular_percentiles`, `procesar_anomalias_temperatura`, ...) behind a thin `__main__` CLI wrapper that just supplies default repo-relative paths. **They were already importable as a library** – calling them directly from a notebook with explicit paths works without modification. This was verified by importing `unir_archivos` and timing `process_yearly_precipitation_data` against a real local `.grib` file before writing the Colab notebook described below.

9.2 `src/drive_sync.py`

A new, small helper module (not part of `aci_lib`) for pulling data out of the shared Google Drive folder (Section 7) into this repo's `data/` layout, for use inside Colab. That folder (`My Drive/2. Datos`, confirmed from the Drive UI – it is a shortcut sitting directly in `My Drive` root; "Indice Climatico Actuarial", seen in Drive's breadcrumb/location panel, is the owner's canonical path, not where the shortcut lives) is shared with specific named collaborators, not "anyone with the link" – confirmed from its "Who has access" panel, which is also why anonymous fetching (tested with `gdown`) returns HTTP 401. So `drive_sync.sync(...)` instead copies (or symlinks) from an already-mounted `google.colab.drive`, which authenticates as whoever is logged into that Colab session.

The full structure of `2. Datos` is now confirmed (via `notebooks/explore_drive_structure.ipynb` in the `aca_aci_collab` repo): `shapefiles/` is flat (48 files, matches `data/shapefiles/`); `era5/completos/` is the flat raw archive (195 files, `era5_{rain,tmp,wind}_<year>.grib` – exactly what `data/raw/era5/` expects); `era5/union/` and `era5/Combined/<date>/` hold Stage 1 merged-daily outputs (differently named, or date-versioned, respectively – not a direct copy target); `era5/percentiles_nc/<date>/` holds dated Stage 2 baseline-percentile snapshots (names *do* match `data/processed/` exactly, but picking "current" means picking a date); `salidas_colombia/` and `salidas_cundinamarca_bogota/` are each a single `.zip`, not a flat folder; and `datos_indice/` is a dated personal archive (`2024-12/`, `2025-06/`, `2025-12/`) of `anomalias_<region>` CSVs that doesn't map onto any one local directory.

`DEFAULT_MAPPING` accordingly only covers the three unambiguous cases – `shapefiles`, `era5/completos` (nested-path keys work: `sync()` just `os.path.join`s them), and the two `salidas_*` zips, which `sync()` detects and extracts automatically rather than copying the `.zip` itself. The dated/renamed subfolders (`union/`, `Combined/`, `percentiles_nc/`, `datos_indice/`) are deliberately left out – picking a date or renaming a file is a judgment call the module does not make for you; pass an explicit `mapping=` for those.

9.3 `aca_opt_benchmark.ipynb`

Lives in the `aca_aci_collab` companion repo's `notebooks/`, alongside `aci_lib_quickstart.ipynb`, but clones *this* pipeline repo rather than the companion repo, since that's what has the scripts to benchmark. It fetches one year of raw temperature `.grib` (not the whole ∼13 GB archive) and times Stages 1–3 against just that year, then extrapolates linearly to a full 1961–2024 (64-year) run for Stages 1 and 3. Stage 2 (baseline percentiles) is timed against the same single year purely as a computational-shape proxy – the real pipeline computes it once over the fixed 30-year 1961–1990 baseline, not once per year, so its extrapolated figure isn't meaningful; multiply the single-year Stage 2 time by roughly 30 instead.

9.4 Verified, then partially reopened: Colab vs. local reproducibility

1-year smoke test (2026-08-10, year 1985, single process): Stage 1 took 31.8s in Colab vs. 22.1s locally, Stage 2 11.7s vs. 8.8s, Stage 3 30.8s vs. 26.6s – different hardware, same order of magnitude. The computed `t_90/t_10` anomaly values matched **bit-for-bit** between the two runs (-0.436035 / -0.737441 for December 1985, identical `NaN` pattern for the other 11 months). An unpinned Colab environment (`xarray==2026.7.0` vs. this repo's `xarray==2024.11.0`) silently dropped the `count_hot/count_cold` columns in this run; pinning every relevant package to `requirements.txt`'s exact versions (`xarray==2024.11.0`, `cfgrib==0.9.14.1`, `rioxarray==0.18.2`, `geopandas==1.0.1`, `netCDF4==1.7.2`, `eccodes==2.39.1`) appeared to resolve it at the time.

Full run (2026-08-10, 1961–2024, 30-year baseline, `use_multiprocessing=True`): with the same pinned versions, the `count_hot/count_cold` columns were *still* missing, and `t_90/t_10` no longer matched exactly (max abs. diff ≈ 0.02, mean ≈ 0.001, against the official `salidas_colombia` reference, 768/768 rows aligned). This contradicts the smoke test's pinning-fixes-it conclusion above – either the pin silently didn't take in that session, or version pinning was never the real cause. **Leading hypothesis, not yet confirmed:** the smoke test ran with `use_multiprocessing=False`; the full run (like the real pipeline's own default) used `multiprocessing.Pool`. `calcular_anomalias_temperatura.py` keeps a module-level `_percentiles_cache` dict wrapping lazily- loaded `xr.open_dataset()` handles – a known-fragile pattern under fork-based multiprocessing, where file handles duplicated across `fork()` can behave inconsistently. Re-running Stage 3 with `use_multiprocessing=False` at full scale would confirm or rule this out; not yet done. Treat both the missing columns and the t_90/t_10 drift as **open, unresolved** until then – do not trust downstream (composite index, multi-region) results computed with `use_multiprocessing=True` without re-checking this.

9.5 `aca_opt_full_replication.ipynb`

Scales `aca_opt_benchmark.ipynb` up from a 1-year smoke test to the real thing: fetches all `era5_tmp_<year>.grib` files (1961–2024) rather than one, runs Stage 1/2 over the actual 30-year 1961–1990 baseline (not a 1-year proxy), Stage 3 over the full record, and then diffs the resulting `anomalies_temperature_combined.csv` against the official `salidas_colombia` output already on Drive (fetched into a separate `_official_reference/` path so it can't be overwritten by the fresh computation) –

column-by-column max/mean absolute difference, not just a visual spot-check. Expected runtime is 45–90 minutes (network fetch of ~5.5 GB plus compute); not something to run casually, but the strongest available evidence that `aca_indice_climatico_opt` genuinely works as an importable library against Drive-sourced data end to end, not just for one hand-picked year.

9.6 `aca_opt_multi_region_monthly.ipynb`

Extends the replication from temperature-only to all four monthly components `calcular_anomalias_regiones.py`'s existing `procesar_anomalias_region()` already orchestrates per region – temperature (T90/T10), wind power (WP), precipitation (Rx5day), and drought (CDD) – across the four regions this repo documents as actually populated (Section 2.2: national, Antioquia, Cundinamarca-Bogotá, Valle del Cauca). No new pipeline logic; this wires already-importable functions to Drive-sourced data across three raw variables (temperature, precipitation, wind) instead of one. Expected runtime is 2.5–3.5 hours for the default 4 regions – substantially longer than the temperature-only replication, since it triples the raw data fetched and runs three anomaly calculations per region.

Two implementation details surfaced while wiring this up, both pre-existing repo quirks, not new bugs:

- `unir_archivos.py`'s three `process_yearly_*_data` functions have **inconsistent parameter order**: `process_yearly_data_tmp` and `process_yearly_precipitation_data` take `(file_path, year, variable, output_dir)`, but `process_yearly_wind_data` takes `(file_path, year, output_dir, variable)` – `output_dir` and `variable` swapped. A generic wrapper assuming one order would silently misdirect wind's output; the notebook uses small per-variable adapters instead of "fixing" the scripts' order.

- `calcular_percentil_lluvia.py`'s `calcular_percentiles()` saves `era5_lluvia_percentil.nc` (singular), but `calcular_anomalias_lluvia.py` reads `era5_lluvias_percentil.nc` (plural) – confirmed as a real, pre-existing mismatch: this repo's own `data/processed/` contains *both* filenames side by side, evidently from someone working around it manually rather than fixing the source. The notebook copies the percentile output under both names rather than silently hiding the inconsistency; fixing the actual name mismatch in the two scripts would be a reasonable, separate follow-up.

A third issue found this way was an actual bug, not a quirk, and was fixed in the source (not worked around): `calcular_anomalias_lluvia.py`'s `process_year()` selected a year's grib file with `[f for f in files if str(year) in f and f.endswith(".grib")][0]` – no variable-name filter, unlike `calcular_anomalias_temperatura.py` and `calcular_anomalias_viento.py`, which both filter further (`"tmp" in f` / `"wind" in f`). As long as `data/raw/era5/` held only rain grib this never mattered; once this notebook fetched all three variables into one directory, it started picking temperature or wind files for most years, failing with `"No variable named 'tp'"`. Caught live during an actual Colab run (2026-08-11); the missing `"rain" in f` filter was added directly to the script (matching the existing pattern in the other two), verified against real data before pushing. The already-published `salidas_colombia` reference was unaffected (768/768 rows present, confirmed by row count) – it was evidently computed back when `data/raw/era5/` held only rain grib, before temperature/wind were added.

9.7 Performance: decode raw grib once per year, not once per region

The first version of this notebook called `procesar_anomalias_region()` once per region, and each call independently re-decoded the same raw grib from scratch – 4 regions × 3 variables × 64 years = 768 redundant grib-decode passes, identical until the final shapefile clip – understandably frustrating given how long a run was taking, since multiprocessing across years does not fix redundant work, and

GPU/TPU runtimes provide *zero* benefit here (nothing in this stack – `xarray`/`cfgrib`/ `pandas` – is CUDA-aware).

Temperature and wind both expose a clean split – `load_annual_grid_data` / `load_annual_grid_data_safe` take an *optional* `shapefile_path`, decoding unclipped data when it's `None` – so the notebook now decodes each year once and clips that one decode to all 4 regions via plain `rioxarray` (`.rio.write_crs(...).rio.clip(...)`), calling `calcular_anomalias`/`calcular_anomalias_viento` per region-month against the shared decode. Multiprocessing still parallelizes over years (unchanged from before), so this combines with, not replaces, the existing 7-worker parallelism. **Verified bit-for-bit identical** against the original per-region code path (year 1985, Colombia, both temperature and wind) before shipping – same values, roughly a 4x reduction in redundant decode work.

9.8 Stage 1 had no parallelism at all – an oversight, not a hardware limit

Separately from the region-level redundancy above, `aca_opt_multi_region_monthly.ipynb`'s (and `aca_opt_full_replication.ipynb`'s) Stage 1 originally processed its 30 baseline years in a plain sequential `for` loop – `unir_archivos.py`'s three `process_yearly_*_data` functions have no cross-year dependency (each year writes a uniquely named intermediate file, unlike drought's CDD in Section 9.9), so this was safely, trivially parallelizable and simply wasn't. This alone accounted for roughly 30 of the multi-region notebook's total minutes. Fixed by parallelizing over years with the same `multiprocessing.Pool` pattern used elsewhere – named top-level task functions per variable, since `Pool` pickles tasks to send to worker processes (even under Colab's `fork` start method) and lambdas aren't picklable. Verified with real `Pool` execution (not just reasoning about it) against 6 real years of local data: 34.2s parallel vs. ~132s sequential, all 6 output files correct.

9.9 Precipitation/drought: a real correctness hazard, not just a performance gap

Precipitation is *not* sped up the same decode-once way temperature and wind are, and drought could not safely be either, for a reason more serious than "not yet done": `calcular_anomalias_lluvia.py`'s `calcular_interpolacion()` (the CDD/drought calculation) reads the **previous year's** saved `datos_maximos_{year-1}.nc` to compute the current year's value, and writes its own for the next year – a sequential, year-over-year dependency. `procesar_anomalias_lluvia()`'s own default is `use_multiprocessing=True`, parallelizing over years via `multiprocessing.Pool` – which does not guarantee execution order. Any year whose predecessor has not yet been written when it runs silently falls back to a cruder approximation (`except FileNotFoundError` in `calcular_interpolacion`) instead of the proper interpolation. **This is a pre-existing risk in this script's own default behavior, not something introduced by this notebook** – and it is plausible, not yet confirmed, that some official reference output was affected, since `use_multiprocessing=True` is the default a plain call would use.

(`load_grid_data`'s `shapefile` parameter was still made properly optional – its docstring already said "if a shapefile is provided, clips the data," but the implementation clipped unconditionally with no `None` guard. That part was a real, narrow, worth-fixing gap; the CDD ordering issue is a different, larger one.)

Given that, `aca_opt_multi_region_monthly.ipynb` parallelizes precipitation/drought across **regions** instead of years: each region gets its own worker process running `procesar_anomalias_lluvia(..., use_multiprocessing=False)` – sequential and therefore correct within that region – with up to 4 regions running concurrently. Real parallelism (up to 4x, one region per core) without touching the ordering question above. Verified by calling the worker function directly (not through `Pool`, since Windows' `spawn` multiprocessing model does not reliably reflect Colab's Linux `fork` model for local testing) against two regions – distinct, plausible output for both, no cross-region collisions.

This leaves the CDD ordering issue itself unresolved – a legitimate follow-up would be making `calcular_interpolacion` not depend on sequential file I/O at all (e.g. computing each year's raw dry-

day count independently and doing the cross-year blend as a separate, explicitly ordered pass), which is a real redesign, not something to bundle into a performance-motivated notebook change.

This changes performance only for temperature/wind, and the region- vs. year-level parallelization choice only for precipitation/drought – the open multiprocessing-correctness question from Section 9.4 (the `count_hot`/`count_cold` and `t_90`/`t_10` drift, specific to temperature) is unaffected by any of this and remains open regardless of which version of this notebook is run.

9.10 Rendering `article1.tex` inline

A cell in `aca_opt_multi_region_monthly.ipynb` compiles `articles/aci_co_submission/article1.tex` – the authoritative methodology paper (Section 2.2), *not* the superseded root-level `articles/article1.tex` – and embeds the resulting PDF inline via a base64 data URI, rather than just offering it for download. This only works because that file (and its figures under `graficas/paper_figures/`) is actually tracked in git – confirmed before building this, since `ARCHITECTURE.md`'s git-tracking-gap section originally listed all of `articles/aci_co_submission/` as untracked; that changed once the pending local commits were pushed (Section 7). `article1.tex` uses a hand-written `thebibliography` block, not `natbib`+`bibtex`+a separate `.bib` file, so two `pdflatex` passes (for cross-references/TOC) compile it with no `bibtex`/`biber` step needed – confirmed by compiling it locally (53 pages, no undefined references, no LaTeX errors) before writing the notebook cell.

9.11 Daily vs. monthly: a cross-validation, not just a resolution comparison

`produce_daily_anomalies.py` (daily, DOY-bootstrap thresholds, following `run_aci_colombia.py`'s algorithm) and `calcular_anomalias_temperatura.py` (monthly, calendar-month thresholds, the OPT pipeline used everywhere else in this notebook) are **two independently-implemented pipelines** kept in sync specifically to cross-check each other (Section 7 on `run_aci_colombia.py`'s original purpose), not two resolutions of the same computation. A new notebook section compares them properly:

- Reuses `run_aci_colombia.py`'s per-year *preprocessing* functions only (`preprocess_temperature_-- year` etc.) – not its full two-phase pipeline – against the same raw grib already fetched for the OPT pipeline, avoiding a second download. Verified against real local data (1961–1962): both years' temperature, wind, and rain preprocessing completed correctly (~88s/year for all three variables combined).
- Compares **unstandardized** exceedance fractions – the OPT pipeline's `count_hot` column against the daily pipeline's `T90` resampled to a monthly mean – not the *standardized* `t_90` column used everywhere else in this notebook, which is not on the same scale as either.
- Scoped to Colombia, temperature only, the 1961–1990 baseline (the DOY thresholds need it) plus a configurable `COMPARISON_YEARS` (default: 2020–2024) – widening it costs preprocessing time linearly.
- `produce_daily_anomalies.py`'s own `doy_percentile` computes each of the 366 day-of-year thresholds as a separate `.isel()`/`.quantile()` xarray call – measured at **150s for one of the four calls needed, on just a 2-year test subset** (would be far worse against the real 30-year baseline). A functionally-identical numpy rewrite (same day-of-year windowing logic, `np.percentile` instead of 366 separate xarray operations) measured **1.6s** on the same input – a 92x speedup, with a 1.83×10^{-5} max difference (floating-point-level, from a different default interpolation method, not a real disagreement). The notebook uses this faster version, defined locally in the cell – `produce_daily_- anomalies.py` itself is unchanged, since this is a notebook-side convenience, not a pipeline correctness fix. The full chain (fast thresholds → `region_mask` → `compute_t90_t10_daily` → `spatial_mean`) was verified

end-to-end against real local data before shipping: 6.9s for all four thresholds, sensible T90 values in the expected $[0, 1]$ range.

9.12 Drive-backed result caching, raw-grib access, and loading outputs back

Every restart this session re-ran Stage 1/2 (the 1961–1990 baseline merge/percentiles) from scratch – ∼20–30 minutes each time, for outputs identical across runs regardless of which regions or `COMPARISON_YEARS` Stage 3 uses. Three additions address this and two adjacent, explicitly-requested needs:

- **Caching** (`drive_sync.cache_upload/ cache_restore/cache_complete`, verified locally – empty-cache detection, upload, complete-cache detection, restore into a fresh directory, partial-cache correctly reported as incomplete): deliberately **file-list-based, not whole-folder**, and deliberately scoped to *only* the 8 small, stable Stage 1/2 baseline files – *not* Stage 3's per-region output, where this session's still-open correctness questions live (Section 9.4, Section 9.9). Caching those silently would risk reusing something still being debugged. Defaults to the user's own `My Drive` space, not the shared `2. Datos` team folder – writing generated cache files into shared storage by default, without teammates expecting them, isn't something to do unprompted.
- **Caching gap closed**: the caching above only covered the OPT pipeline's Stage 1/2 baseline – Section 9.11's `aci_daily/` preprocessing (a separate intermediate format nothing else uses) was still unconditionally recomputing all 35 years (30 baseline + 5 `COMPARISON_YEARS`) every run, ∼50 minutes, even after the baseline caching above was added. Fixed with the same primitives, scoped separately and **per year** rather than all-or-nothing: whichever years were preprocessed in *any* previous run are restored and skipped (via the existing per-file `if not exists` check inside the preprocessing task, unchanged), only genuinely new years cost anything. Verified with a synthetic cache-then-restore-then-compute-then-upload test (2 pre-cached years correctly restored and skipped, 1 new year correctly computed, all 9 resulting files correctly persisted) before shipping.
- **Raw grib access** (`load_raw_grib()`): a thin wrapper around `xr.open_dataset(..., engine="cfgrib")` plus optional shapefile clipping, for exploration outside the fixed pipeline – verified against real data, both unclipped (71×68 full extent) and clipped to a small region (Valle del Cauca, correctly reduced to 8×6).
- **Loading outputs for further analysis**: rather than reimplementing a browse/load layer, the notebook clones `aca_aci_collab` and points `aci_lib` (Section 8) directly at the session's own `data/processed/` – what gets loaded is what this run just computed, not a separately-fetched copy.

9.13 `count_hot/count_cold` dilution bug (found via Section 9.11, fixed in the pipeline)

The daily-vs-monthly comparison (Section 9.11) surfaced a persistent $\sim 2\times$ gap between the two pipelines' unstandardized T90 fraction that did *not* shrink or vanish under several hypotheses tested against real local data: neither a DOY-window/leap-year artifact, nor `produce_daily_anomalies.py`'s use of a 10pm–6am `night_max` window instead of article1.tex's true daily-minimum TN90p (confirmed as a genuine definitional difference from the paper, but recomputing with the correct full-day min made no difference to the gap), nor `region_mask()`'s `all_touched=True` clipping parameter (changes the result by $< 2\%$), nor a stale cached `aci_daily` file (byte-identical to a fresh regeneration).

The actual cause: `calcular_anomalias_temperatura.py`'s `calcular_anomalias()` clips to Colombia via `shape.rio.clip(..., drop=True)`, which crops to the shapefile's *bounding box*, not its polygon – for January, the cropped grid has 3,283 cells but only 1,493 are actually inside Colombia (the rest are Pacific/Caribbean ocean and Venezuela/Panama/Ecuador/Brazil border padding,

NaN in the temperature data). The line `count_above_90_max = (daily_max > percentile_90_max) .astype(int)` then silently destroys that: `NaN > x` evaluates to `False` under IEEE754, and `.astype(int)` turns that `False` into a real `0`, indistinguishable from "inside Colombia, didn't exceed." The final `anomalies.mean(dim=['latitude','longitude'])` then divides by the full padded box (3,283) instead of the true polygon (1,493) – diluting `count_hot`/`count_cold` by roughly that ratio ($\approx 2.2\times$). `calcular_anomalias_viento.py`'s `compute_occurrences()` (WP's raw exceedance count) has the identical pattern.

Verified with real data before concluding this was the cause (not just plausible): the raw hourly grib values, the full-month gridded `day_max`, and the P90 thresholds were all confirmed **bit-for-bit identical** between the two pipelines (zero difference, checked cell-by-cell across the whole grid) – ruling out any data or threshold discrepancy. Recomputing OPT's January 2020 TX90p by hand, correctly restricted to only the 1,493 real Colombia cells, reproduced the daily pipeline's ≈ 0.39, not OPT's diluted 0.175. So the daily pipeline's numbers were correct all along; `count_hot` was the one silently wrong.

The standardized columns are unaffected, for a reason grounded in the code rather than a coincidence: `t_90`/`t_10` (`anomalies_above_max` etc.) get their `NaN` mask from `mean_max`/`std_dev_max` – computed by `calcular_percentil_temperatura.py` per grid cell, never spatially collapsed, and clipped *separately* from the count arrays – so they were already correctly excluding out-of-polygon cells regardless of the count-dilution bug. Confirmed directly: recomputing Jan 2020 after the fix gives `t_90 = 2.781`, `t_10 = -0.8136` – bit-identical to the pre-fix values – while `count_hot` moves from 0.1988 to 0.4373 (matching the daily pipeline's independently-computed 0.4373 almost exactly) and `count_cold` from 0.0153 to 0.0337. Since the ACI-CO composite and article1.tex's headline results are built from the standardized columns, they are not expected to change from this fix; only the raw `count_hot`/`count_cold`/`count_above` columns (and anything reading them directly, like the notebook's §10 comparison) were wrong.

Fix: both scripts' exceedance-count computation was changed from `.astype(int)` to `xr.where(cond, 1.0, 0.0).where(source.- notnull())`, preserving `NaN` through to the final spatial mean so `skipna=True` (xarray's default) correctly excludes out-of-polygon cells. This is a source-code fix in the pipeline repo, not a notebook workaround – the multi-region notebook imports these functions directly (`import calcular_anomalias_temperatura as m; m.- calcular_anomalias(...)`), so it inherits the fix automatically on its next run without any notebook change. One consequence: the `salidas_*` official reference on Drive (Section 9.11's §8b comparison) predates this fix, so a fresh run will show `count_hot`/`count_cold`/`count_above` diverging from that reference by the old $\sim 2.2\times$ factor until the reference is regenerated – `t_90`/`t_10`/`anomalies_above` should still match closely, as before.

9.14 CUDA: where it actually applies, and where it can't

Requested explicitly after Section 9.4's and this document's own profiling kept showing that GPU compute wasn't the bottleneck. Profiled with real local data before writing any GPU code, to know what a CUDA port could and couldn't touch:

- **GRIB decode + day/night resample**: **43.8s for one year** of temperature alone (measured locally, real 2020 data) – across a 30-year baseline that's $\approx$22 minutes for temperature, with wind and rain adding more. This runs through `cfgrib`/`eccodes`, a CPU-only C library with no CUDA path – porting it isn't an option without replacing the decoder entirely.
- **Shapefile clipping** (`rioxarray`/GDAL): also CPU-only, no GPU-accelerated equivalent that fits this stack.
- **DOY-window percentile computation**: the one piece that *is* portable – **21.0s for the full 30-year baseline** (measured locally, real data, the vectorized numpy version from Section 9.11).

Decode alone outweighs the portable math by roughly 60x for temperature; including wind and rain decode, more. A full "port the pipeline to CUDA" was explicitly not attempted for this reason – it would mean rewriting around `cfgrib` and GDAL entirely, a much larger and riskier undertaking than the actual payoff (a few tens of seconds off a run that takes tens of minutes) justifies.

What *was* shipped: `doy_percentile_fast` in the multi-region notebook's §10 (Section 9.11) now auto-detects a CUDA device via `cupy` (`cupy.cuda.is_available()`) and runs the percentile computation on GPU when one is present, falling back to plain numpy otherwise – same code path either way (`xp = cp if GPU else np`), so no separate CPU/GPU implementation to keep in sync. Verified locally (no CUDA device on this machine) that the fallback path is **bit-identical** (0.0 max diff) to the existing numpy implementation before shipping; the actual GPU speedup itself can only be measured by running it on Colab's GPU runtime, since no local GPU was available to benchmark against. A cell before it installs `cupy-cuda12x` (falling back to `cupy-cuda11x`) only if `cupy` isn't already importable, so it's a no-op on Colab GPU images that ship it preinstalled.

9.15 §13: third-party CUDA GRIB decoder – explicitly unverified

Requested after Section 9.14 – specifically, to try brycetur21/grib_- decoder_gpu, found and linked by the user. Read before running: this is **the one section of this notebook that was not, and could not be, verified against real data before shipping** – every other addition in this document was tested locally first (see the "Verified" callouts throughout Sections 9.11 and 9.13); this one couldn't be, since it needs a CUDA device this machine doesn't have, and building the CPU path alone needs Linux packages (`libeccodes-dev`, `libhdf5-dev`) not readily available here either.

What was checked instead – cloning the repository and reading its actual source (1,453 lines across `src//include/`) before writing any notebook code, rather than trusting its README's framing:

- **0 stars, 0 forks, 1 watcher, 14 commits, single contributor** – no external validation of correctness.
- **It measures timing only.** `Scheduler::process_gribs()` (`src/scheduler.cu`) prints `"CPU/GPU processing took ..."` and returns nothing – no NetCDF, no array, nothing written to disk. There is no output format for this pipeline (or any pipeline) to consume, regardless of whether it builds or runs correctly.
- **Its `"both"` (CPU+GPU simultaneous) mode is broken**: `cpu_files`/`gpu_files` are declared but the code that would populate them from `filenames` is commented out, so that mode processes two empty lists. The notebook only exercises `cpu`/`gpu` modes separately for this reason.
- **Still depends on `libeccodes`** for lat/lon metadata (`load_lats_lons_grib`) – confirms the earlier suspicion in Section 9.14 that this isn't a full GRIB-decode replacement; only the bit-unpacking (`BitGroup`/`BitReader`) plausibly runs on GPU.
- **Compiled for a specific architecture**: `CUDA_ARCHITECTURES 80` in `CMakeLists.txt` (Ampere/SM80) – matches Colab's A100, likely fails to run on a T4 or other non-Ampere runtime.
- **File discovery is hardcoded**: `get_grib_files()` only matches a literal `.grib2` extension in a hardcoded `../grib_data` path – this repo's raw archive is `era5_*.grib` (no `2`), so the notebook copies (never renames in place) a handful of real files into what the tool expects, rather than touching the pipeline's own raw data.

The notebook cell installs prerequisites, clones the repo, stages sample files, builds with `cmake`/`make`, and if the binary exists afterward, runs `-device=cpu` then `-device=gpu` sequentially (`"both"` skipped as broken). If the build fails, the notebook says so explicitly rather than silently doing nothing – that

would mean this tool needs setup beyond what its README documents, not that anything in the actual pipeline is broken. **This section does not feed into, and cannot feed into, any other part of the pipeline** – it exists purely to let the tool's claims be checked directly in an environment (Colab, with a real GPU) that can actually run it, something neither this machine nor Claude could do first.

9.16 Raw-grib transfer: many small files vs. one archive

Prompted by a Reddit thread the user linked directly – r/Machine- Learning, "Best way to transfer large datasets to Colab" – whose OP hit an identical symptom to this project's own earlier "way too slow" complaints: "Took me an hour just to glob through which files were in google drive." Their own resolution, echoed by several top comments (`seba07`, `somnet`, `Schorsi`): zip the dataset once, transfer the single archive, unzip locally – copying many small files off a mounted Drive pays large per-file overhead that a single larger transfer avoids.

§3's fetch of `era5/completos` was exactly the pattern being warned about: 195 loose `.grib` files, each copied individually via `shutil.copy` in a loop. Unlike `salidas_colombia/` `salidas_cundinamarca_bogota` (already single `.zip`s on Drive, already auto-extracted by `sync()`), the raw archive had no single-file fast path at all – and, unlike the Stage 1/2 baseline caching (Section 9.11's caching-gap discussion), this fetch runs on effectively *every* session, since it's raw input, not cacheable derived output.

`drive_sync.archive_and_cache_raw()`/`restore_raw_- archive()` apply the same fix as the existing baseline cache, adapted for many-small-files: zip *locally* (fast – local disk, no network-mount overhead) rather than zip-over-the-mount, upload the single resulting archive to the user's own Drive cache space, and on a later run check the archive's directory listing (metadata only, no decompression) against the currently-requested `YEARS` before trusting it – returns a cache miss rather than silently extracting a partial dataset if `YEARS` has since widened. §3 tries this restore first; only on an actual cache miss does it fall back to the original per-file loop, then archives the result for next time. `ZIP_STORED` (no compression) is used deliberately – GRIB2 already applies its own internal packing, so DEFLATE would add real CPU cost for little size reduction; the win here is one file transferred instead of 195, not fewer bytes.

Verified against real local data (3 real 1961–1963 temperature grib files) before shipping: round-tripped through archive-then-restore **bit-identical** (SHA-256 match), a request for a filename outside the cached archive correctly reports a miss instead of a partial extraction, a missing archive correctly reports a miss, and the local zip is deleted after upload rather than left behind as a duplicate multi-GB file. The actual Drive-transfer speedup itself – unlike the round-trip correctness above – could not be measured locally, for the same reason as Section 9.14's GPU speedup: there is no mounted Drive to benchmark against outside Colab.

Revised after a real run: per-variable, not all-or-nothing. The first version above cached one archive covering all three variables – meaning nothing got cached until all 192 files (64 years × 3 variables) had been fetched, with no progress output printed in between. In practice this looked identical to a hung cell and the user stopped the runtime partway through, which under Colab's ephemeral `/content` filesystem discarded *everything* fetched so far, including progress toward variables that would otherwise have already finished. Changed to cache **one archive per variable** (`era5_completos_{tmp,rain,wind}.zip`) with a progress line printed every 10 years, using the same `archive_name` parameter `archive_and_-cache_raw()`/`restore_raw_archive()` already exposed (no changes needed there) – so a variable that finishes and gets cached before an interruption stays cached, and the per-file fetch loop itself skips files already present locally (not just those already archived) so a resumed run within the same session doesn't re-copy what it already has. Verified with the same round-trip test as above, exercising the (previously untested) custom `archive_name` path specifically: two variables cached under separate archive names, restored independently, bit-identical, and a request against a never-cached third archive name correctly

misses rather than erroring.

9.17 Caching Stage 3's decode step, not just the raw-file fetch

The raw-archive caching above (Section 9.16) fixed getting the raw `.grib` files into Colab quickly; it did nothing for what happens to them once there. A real run showed §6a (temperature, all 4 regions) taking **53.6 minutes** – confirming that `load_annual_grid_data()`/`load_annual_grid_- data_safe()` (the `cfgrib` decode) was being re-run from scratch every session, matching this document's own earlier profiling (Section 9.14: 43.8s for one year of temperature decode alone). Measured again here with a cleaner, decode-only test: **20.2s to decode one year of temperature**, **0.8s to reload the same year from a cached NetCDF** – roughly a 25x reduction on a cache hit. Wind: 44.6s decode vs. 0.0s reload (wind's decode already resamples to daily mean internally, making the cached file tiny – 5.6 MB/year vs. 75 MB/year for temperature's hourly data).

Design: cache the decode step, not the science. `calcular_anomalias()`/`calcular_- anomalias_viento()` already accept pre-loaded xarray data as an argument – they don't care how it was loaded. So §6a's `_year_task_temp`/`_year_- task_wind` now check a local cache before calling `load_annual_grid_data`: on a hit, reload the cached NetCDF (fast); on a miss, decode as before, then write the result to the cache *and* upload it via `drive_sync.cache_upload()` before proceeding. **Zero changes to** `calcular_anomalias_- temperatura.py`/`calcular_anomalias_viento.py` – this caches only the decode step that already fed them, so it has no interaction with the still-open multiprocessing-correctness question (Section 9.4) that is the reason Stage 3's actual per-region *output* is deliberately not cached.

Per year, uploaded immediately – not batched. Directly informed by what happened with the raw-archive cache (Section 9.16's "revised after a real run" note): a 53-minute stage is long enough to plausibly get interrupted, and an all-or-nothing archive built only after every year finishes would lose everything in that case. Each worker task uploads its own year's cache file as soon as that year's decode completes, inside the `multiprocessing.Pool` task itself (safe under Colab's fork start method – the mounted Drive filesystem is inherited by forked workers, no per-worker re-authentication needed, and each year writes to a uniquely-named file so there's no cross-worker write conflict). A cache-write/upload failure is caught separately from a decode failure – the year's data is already successfully decoded and in memory at that point, so a Drive hiccup costs that year's caching benefit next time, not this run's result.

Verified against real local data before shipping. Beyond the raw array round-trip (bit-identical values and time coordinates, confirmed above), the check that actually matters: decode-fresh vs. load-from-cache were run through the complete, unmodified `calcular_anomalias()`/`calcular_anomalias_- viento()` call chain (shapefile clip, monthly extraction, percentile comparison, standardization) for a real month (June 2021, Colombia), and every output column compared. Temperature: `t_90`, `t_10`, `count_hot`, `count_cold` all bit-identical between the two paths. Wind: `count_above`, `anomalies_- above` both bit-identical. This is the same standard applied to the `count_hot` dilution fix (Section 9.13) – confirming the cache doesn't just store the right bytes, but that everything downstream of it produces exactly the same science.

10 Cleanup priorities (standing list)

Ordered roughly by urgency/impact; see `ARCHITECTURE.md` for the canonical, living version of this list.

1. **Secrets**: rotate the Earth Data Hub PAT tokens and the CDS key (Section 7.1); scrub from git history if the two committed copies need to be fully removed.
2. **Commit the real pipeline**: the untracked files listed in Section 7.
3. **Purge committed generated junk**: build artifacts and data files committed at the repo root instead of under `data/`.
4. **Fix `.gitignore`**: currently wraps the real ignore rules in a literal PowerShell heredoc command that got committed instead of its output.
5. **Delete disk-only bloat** (not committed, safe to remove locally): environment archives, unrelated test data, `__pycache__/`.
6. **Consolidate duplicated helpers** into `src/utils/` (mostly done – see `get_available_years`, `get_cached_shapefile`).
7. **Resolve naming/content mismatches**: `display.py`, `sealevel2.py`, stale internal docstrings.
8. **Decide the fate of superseded scripts**: `graficas_altaresolucion.py`, `docs/data_dictionary.md`, root `articles/article1.tex`.
9. **Fill or drop the empty doc stubs** in `docs/`.
10. **Populate or drop** the empty `data/processed/anomalias_{bogota,medellin,san_andres_providencia}/` directories and the unused `data/indices/`, `data/zones/` locations.

Confirm scope with a maintainer before acting on any of these – several things that look like dead scratch code are load-bearing work that simply never got committed (Section 7).